

Relatório Elaboração do Projeto

Título do Projeto

MWRP-BOT – Sistema de Gestão para Servidor Discord

Data: 19 de Junho de 2026

Autor: Carlos Madaleno

Sumário

1. Introdução
2. Descrição do Projeto
3. Levantamento de Requisitos
4. Análise de Sistemas
5. Design do Sistema
6. Implementação
7. Testes
8. Implantação
9. Conclusão
10. Anexos

1. Introdução

Contextualização

Este relatório documenta o desenvolvimento do projeto "MWRP-BOT", um bot desenvolvido em Python para a plataforma Discord.

O objetivo do sistema é automatizar tarefas administrativas e de suporte dentro de um servidor Discord, permitindo uma gestão mais eficiente dos utilizadores, tickets de suporte e candidaturas para whitelists.

Objetivo do Relatório

Apresentar todas as fases de desenvolvimento do projeto, incluindo análise, planeamento, implementação, testes e resultados obtidos.

2. Descrição do Projeto

Visão Geral

O MWRP-BOT foi desenvolvido para auxiliar a administração de servidores Discord através da automatização de processos comuns.

O sistema permite:

- Atribuição automática de cargos.
- Gestão de tickets de suporte.
- Gestão de candidaturas para whitelist.
- Registo de logs administrativos.
- Geração automática de transcrições.

Escopo

Incluso

- Sistema de Autorole.
- Sistema de Tickets.
- Sistema de Logs.
- Sistema de Whitelists.
- Transcrição automática de tickets.
- Configuração dinâmica através de comandos.

Não Incluso

- Dashboard Web.
- Integração com FiveM.
- Sistema de Economia.
- Integração com APIs externas.

Objetivos

- Automatizar processos administrativos.
- Melhorar o suporte aos utilizadores.
- Organizar candidaturas de whitelist.
- Facilitar a gestão do servidor Discord.

3. Levantamento de Requisitos

Metodologia de Recolha

Os requisitos foram definidos com base nas necessidades observadas em servidores Discord dedicados a Roleplay.

Requisitos Funcionais

1. Atribuir automaticamente cargos a novos membros.
2. Permitir a abertura de tickets.
3. Permitir o encerramento de tickets.
4. Gerar logs de ações administrativas.
5. Gerar transcrições automáticas.
6. Permitir a criação de candidaturas para whitelist.
7. Permitir aprovação e rejeição de candidaturas.

Requisitos Não Funcionais

1. Interface intuitiva.
2. Facilidade de configuração.
3. Armazenamento persistente dos dados.
4. Segurança das permissões.
5. Estrutura modular e escalável.

4. Análise de Sistemas

Casos de Uso

Administrador

- Configurar autorole.
- Configurar logs.
- Configurar tickets.
- Configurar whitelists.
- Aprovar candidaturas.
- Rejeitar candidaturas.

Utilizador

- Abrir ticket.
- Fechar ticket.
- Criar candidatura.

Análise de Riscos

Risco 1

Perda de dados da base de dados.

Mitigação:

- Utilização de SQLite.
- Possibilidade de backups periódicos.

Risco 2

Configuração incorreta por administradores.

Mitigação:

- Comandos de validação.
- Mensagens de confirmação.

5. Design do Sistema

Arquitetura

O sistema foi desenvolvido utilizando uma arquitetura em camadas.

Presentation Layer

- Commands
- Events

Business Logic Layer (BLL)

Responsável pela lógica de negócio.

Data Access Layer (DAL)

Responsável pelo acesso à base de dados.

Database Layer

SQLite para armazenamento persistente.

Estrutura do Projeto

src/

- commands/
- events/
- bll/
- dal/
- database/

Base de Dados

Tabela Settings

- guild_id
- autorole_id
- logs_channel_id

Tabela Ticket_Settings

- guild_id
- ticket_type
- category_id
- staff_role_id

Tabela Whl_Settings

- guild_id
- whl_type
- category_id
- staff_role_id

6. Implementação

Tecnologias Utilizadas

Linguagem

- Python

Bibliotecas

- discord.py
- sqlite3
- python-dotenv

Ferramentas

- Visual Studio Code
- Git
- GitHub

Estrutura de Código

main.py

Ponto de entrada do sistema.

commands

Contém todos os comandos do bot.

events

Eventos executados automaticamente.

bll

Implementação da lógica de negócio.

dal

Acesso aos dados.

database

Gestão da base de dados SQLite.

7. Testes

Testes Realizados

Autorole

Resultado: Sucesso

Sistema de Tickets

Resultado: Sucesso

Criação de Ticket

Resultado: Sucesso

Encerramento de Ticket

Resultado: Sucesso

Logs

Resultado: Sucesso

Transcrição

Resultado: Sucesso

Sistema de Whitelists

Resultado: Sucesso

Aprovação e Rejeição

Resultado: Sucesso

8. Implantação

Ambiente

Servidor Discord de testes.

Atividades

- Configuração do bot.
- Configuração das permissões.
- Criação de categorias.
- Configuração de cargos.
- Testes finais.

Formação

Foi elaborado um conjunto de comandos administrativos para facilitar a utilização do sistema.

9. Conclusão

O projeto foi concluído com sucesso, cumprindo os objetivos inicialmente definidos.

O sistema desenvolvido permite automatizar tarefas administrativas importantes num servidor Discord, reduzindo o trabalho manual e melhorando a organização dos processos de suporte e candidaturas.

Próximos Passos

- Integração com FiveM.
- Dashboard Web.
- Sistema avançado de candidaturas.
- Sistema de estatísticas.
- Sistema de moderação.

Reflexão

O desenvolvimento do projeto permitiu consolidar conhecimentos em Python, programação orientada a objetos, arquitetura em camadas, bases de dados SQLite e integração com APIs externas.

10. Anexos

- Código-fonte do projeto.
- Estrutura da base de dados.
- Capturas de ecrã do sistema.
- Casos de teste realizados.
- Repositório GitHub.